

Load and display images from the Internet

1. Welcome
2. App overview
3. Display an internet image
4. Display a grid of images with a RecyclerView
5. Add error handling in RecyclerView
6. Solution code
7. Summary
8. Learn more

1. Welcome

Introduction

In the previous codelab, you learned how to get data from a web service and parse the response into a Kotlin object. In this codelab, you build on that knowledge to load and display photos from a web URL. You also revisit how to build a `RecyclerView` and use it to display a grid of images on the overview page.

Prerequisites

- How to create and use fragments.
- How to retrieve JSON from a REST web service and parse that data into Kotlin objects using the [Retrofit](#) and [Moshi](#) libraries.
- How to construct a grid layout with a `RecyclerView`.
- How `Adapter`, `ViewHolder`, and `DiffUtil` work.

What you'll learn

- How to use the [Coil](#) library to load and display an image from a web URL.
- How to use a `RecyclerView` and a grid adapter to display a grid of images.
- How to handle potential errors as the images download and display.

What you'll build

- Modify the `MarsPhotos` app to get the image URL from the Mars data, and use `Coil` to load and display that image.
- Add a loading animation and error icon to the app.
- Use a `RecyclerView` to display a grid of Mars images.

- Add status and error handling to the `RecyclerView`.

What you need

- A computer with a modern web browser, such as the latest version of [Chrome](#).
- Internet access on your computer.

2. App overview

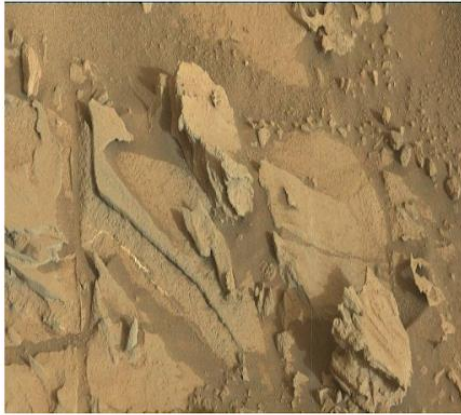
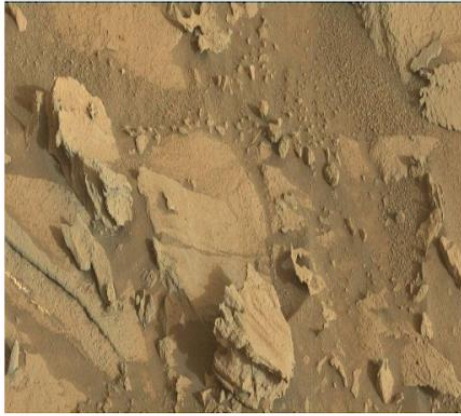
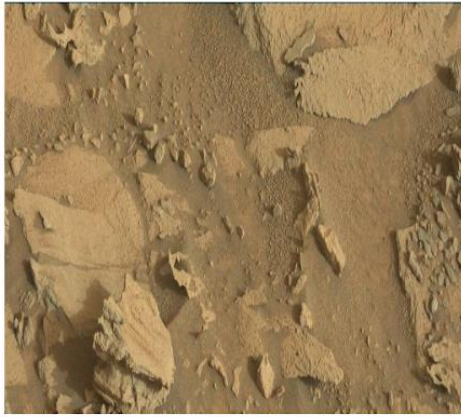
In this codelab, you will continue working with the app from the previous codelab called MarsPhotos. The MarsPhotos app connects to a web service to retrieve and display the number of Kotlin objects retrieved using Retrofit. These Kotlin objects contain the URL's of the real-life photos from Mars surface captured from NASA's Mars rovers.

The version of the app you will build in this codelab fills in the overview page, which displays Mars photos in a grid of images. The images are part of the data that your app retrieved from the Mars web service. Your app will use the Coil library to load and display the images, and a `RecyclerView` to create the grid layout for the images. Your app will also handle network errors gracefully.

11:52



Mars Photos



3. Display an internet image

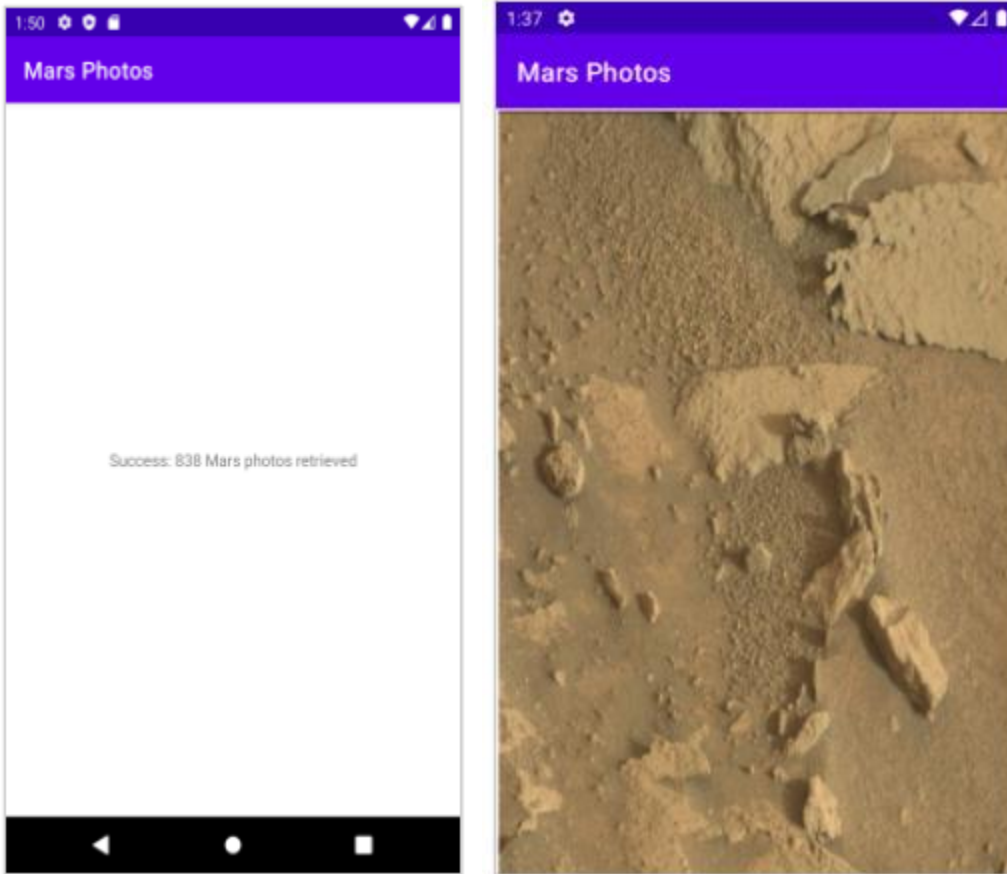
Displaying a photo from a web URL might sound straightforward, but there is quite a bit of engineering to make it work well. The image has to be downloaded, internally stored, and decoded from its compressed format to an image that Android can use. The image should be cached to an in-memory cache, a storage-based cache, or both. All this has to happen in low-priority background threads so the UI remains responsive. Also, for best network and CPU performance, you might want to fetch and decode more than one image at once.

Fortunately, you can use a community-developed library called [Coil](#) to download, buffer, decode, and cache your images. Without the use of Coil, you would have much more work to do.

Coil basically needs two things:

- The URL of the image you want to load and display.
- An `ImageView` object to actually display that image.

In this task, you learn how to use Coil to display a single image from the Mars web service. You display the image of the first Mars photo in the list of photos that the web service returns. Here are the before and after screenshots:



Add Coil dependency

1. Open the [MarsPhotos solution](#) app from the previous codelab.
2. Run the app to see what it does. (It shows the total number of Mars photos retrieved).
3. Open **build.gradle (Module: app)**.
4. In the `dependencies` section, add this line for the Coil library:

```
// Coil
implementation "io.coil-kt:coil:1.1.1"
```

Check and update the latest version of the library from the [Coil](#) documentation page.

5. Coil library is hosted and available on the `mavenCentral()` repository. In **build.gradle (Project: MarsPhotos)** add `mavenCentral()` in the top `repositories` block.

```
repositories {
    google()
    mavenCentral()
}
```

6. Click **Sync Now** to rebuild the project with the new dependency.

Update the ViewModel

In this step, you will add a `LiveData` property to the `OverviewViewModel` class to store the Kotlin object received, `MarsPhoto`.

1. Open `overview/OverviewViewModel.kt`. Just below the `_status` property declaration, add a new mutable property called `_photos`, of the type `MutableLiveData` that can store a single `MarsPhoto` object.

```
private val _photos = MutableLiveData<MarsPhoto>()
```

Import the `com.example.android.marsphotos.network.MarsPhoto` when requested.

2. Just below the `_photos` declaration, add a public backing field called `photos` of the type, `LiveData<MarsPhoto>`.

```
val photos: LiveData<MarsPhoto> = _photos
```

3. In the `getMarsPhotos()` method, inside the `try{}` block, find the line that sets the data retrieved from the web service to `listResult`.

```
try {  
    val listResult = MarsApi.retrofitService.getPhotos()  
    ...  
}
```

4. Assign the first Mars photo retrieved to the new variable `_photos`. Change the `listResult` to `_photos.value`. Assign the first photos url at the index 0. This will throw an error, you will fix it later.

```
try {  
    _photos.value = MarsApi.retrofitService.getPhotos()[0]  
    ...  
}
```

5. In the next line, update the `status.value` to the following. Use the data from the new property instead of `listResult`. Display the first image URL from the photos List.

```
try {  
    ...  
    _status.value = "    First Mars image URL : ${_photos.value!!.imgSrcUrl}"  
}
```

6. The complete `try{}` block now looks like this:

```
try {  
    _photos.value = MarsApi.retrofitService.getPhotos()[0]
```

```
        _status.value = "    First Mars image URL : ${_photos.value!!.imgSrcUrl}"  
    }
```

7. Run the app. The `TextView` now displays the URL of the first Mars photo. All you've done so far is set up the `ViewModel` and the `LiveData` for that URL.

3:42



Mars Photos

First Mars photo image URL : http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000MR0044631300503690E01_DXXX.jpg

Use Binding Adapters

Binding Adapters are annotated methods used to create custom setters for custom properties of your view.

Usually when you set an attribute in your XML using the code: `android:text="Sample Text"`. The Android system automatically looks for a setter with the same name as the `text` attribute, which is set by the `setText(String: text)` method. The `setText(String: text)` method is a setter method for some views provided by the Android Framework. Similar behavior can be customized using the binding adapters; you can provide a custom attribute and custom logic that will be called by the Data binding library.

Example:

To do something more complex than simply calling a setter on the Image view, that sets an drawable image. Consider loading images off the UI thread (main thread), from the internet. First, choose a custom attribute to assign the image to an `ImageView`. In the following example it is `imageUrl`.

```
<ImageView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    app:imageUrl="@{product.imageUrl}"/>
```

If you do not add any further code, the system would look for a `setImageUrl(String)` method on `ImageView` and not find it, throwing an error because this is a custom attribute not provided by the framework. You have to create a way to implement and set the `app:imageUrl` attribute to the `ImageView`. You will use Binding adapters (annotated methods) to do this.

Example of a Binding Adapter:

```
@BindingAdapter("imageUrl")
fun bindImage(imgView: ImageView, imgUrl: String?) {
    imgUrl?.let {
        // Load the image in the background using Coil.
    }
}
```

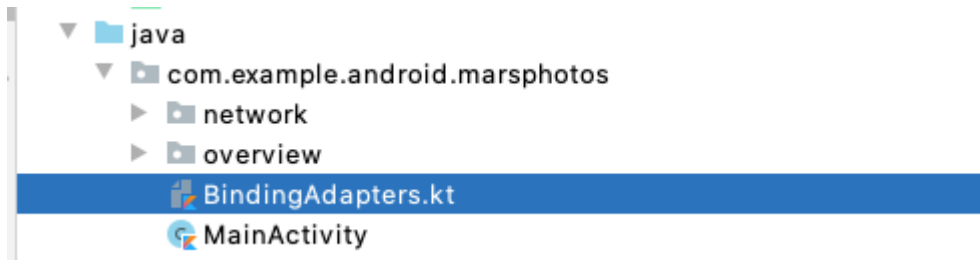
The `@BindingAdapter` annotation takes the attribute name as its parameter.

In the `bindImage` method, the first method parameter is the type of the target View and the second is the value being set to the attribute.

Inside the method, the Coil library loads the image off the UI thread and sets it into the `ImageView`.

Create a binding adapter and use Coil

1. In `com.example.android.marsphotos` package, create a Kotlin file called `BindingAdapters`. This file will hold the binding adapters that you use throughout the app.



2. In `BindingAdapters.kt` create a `bindImage()` function as a [top-level function](#) (not inside a class) that takes an `ImageView` and a `String` as parameters.

```
fun bindImage(imgView: ImageView, imgUrl: String?) {  
  
}
```

Import `android.widget.ImageView` when requested.

3. Annotate the function with `@BindingAdapter`. The `@BindingAdapter` annotation tells data binding to execute this binding adapter when a `View` item has the `imageUrl` attribute.

```
@BindingAdapter("imageUrl")  
fun bindImage(imgView: ImageView, imgUrl: String?) {  
  
}
```

Import `androidx.databinding.BindingAdapter` when requested.

let scope function

`let` is one of Kotlin's [Scope](#) functions which lets you execute a code block within the context of an object. There are five Scope functions in Kotlin, refer to the [documentation](#) to learn more.

Usage:

`let` is used to invoke one or more functions on results of call chains.

The `let` function along with safe call operator (`?.`) is used to perform a null safe operation on the object. In this case, the `let` code block will only be executed if the object is not null.

4. Inside the `bindImage()` function, add a `let{}` block to the `imgUrl` argument, using the safe call operator.

```
imgUrl?.let {  
}
```

5. Inside the `let{}` block, add the following line to convert the URL string to a `Uri` object using the `toUri()` method. To use the HTTPS scheme, append `buildUpon.scheme("https")` to the `toUri` builder. Call `build()` to build the object.

```
val imgUri = imgUrl.toUri().buildUpon().scheme("https").build()
```

Import `androidx.core.net.toUri` when requested.

6. Inside `let{}` block, after the `imgUri` declaration, use the `load(){}` from [Coil](#), to load the image from the `imgUri` object into the `imageView`.

```
imageView.load(imgUri) {  
}
```

Import `coil.load` when requested.

7. Your complete method should look like below:

```
@BindingAdapter("imageUrl")  
fun bindImage(imageView: ImageView, imgUrl: String?) {  
    imgUrl?.let {  
        val imgUri = imgUrl.toUri().buildUpon().scheme("https").build()  
        imageView.load(imgUri)  
    }  
}
```

Update the layout and fragments

In the previous section, you used the Coil image library to load the image. To see the image on screen, the next step is to update the `ImageView` with the new attribute to display a *single* image.

Later in the codelab you will use `res/layout/grid_view_item.xml` as the layout resource file each grid item in the `RecyclerView`. In this task, you will use this file temporarily to display the image using the image URL that you retrieved in the previous task. Temporarily, you will be using this layout file in place of `fragment_overview.xml`.

Note: In this task you will be using `grid_view_item.xml` layout to display a single image temporarily. It is done this way in order to avoid creating and deleting temporary layout files. The generated binding class name will be `GridViewItemBinding`, because the name of the class is based on the name of the layout file which is `grid_view_item.xml`. Even though the binding

class is called `GridViewItemBinding`, you will not be using it to display grid images inside the `RecyclerView` in this task. This is done in a later task.

1. Open `res/layout/grid_view_item.xml`.
2. Above the `<ImageView>` element, add a `<data>` element for the data binding, and bind to the `OverviewViewModel` class:

```
<data>
    <variable
        name="viewModel"
        type="com.example.android.marsphotos.overview.OverviewViewModel" />
</data>
```

3. Add an `app:imageUrl` attribute to the `ImageView` element to use the new image loading binding adapter. Remember that the `photos` contain a list `MarsPhotos` retrieved from the server. Assign the first entry URL to the `imageUrl` attribute.

```
<ImageView
    android:id="@+id/mars_image"
    ...
    app:imageUrl="@{viewModel.photos.imgSrcUrl}"
    ... />
```

4. Open `overview/OverviewFragment.kt`. In the `onCreateView()` method, comment out the line that inflates the `FragmentOverviewBinding` class and assigns it to the binding variable. You will see errors due to removing this line. This is only temporary; you'll fix them later.

```
//val binding = FragmentOverviewBinding.inflate(inflater)
```

5. Use `grid_view_item.xml` instead of `fragment_overview.xml`. Add the following line to inflate the `GridViewItemBinding` class instead.

```
val binding = GridViewItemBinding.inflate(inflater)
```

Import `com.example.android.marsphotos.databinding.GridViewItemBinding` if requested.

Note: This change may result in data-binding errors in Android Studio. To resolve those errors, you need to clean and rebuild the project. Select **Build > Clean Project** and then **Build > Rebuild Project**

6. Run the app. Now you should see a single Mars image.

1:37



Mars Photos



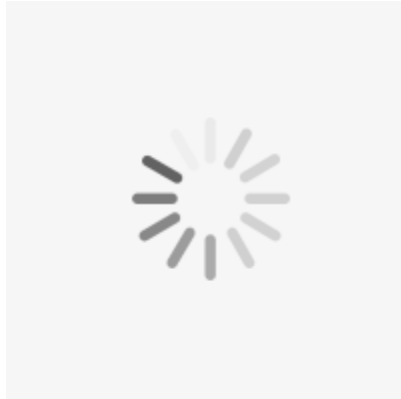
Add loading and error images

Using Coil you can improve the user experience by showing a placeholder image while loading the image and an error image if the loading fails, for example if the image is missing or corrupt. In this step, you will add that functionality to the binding adapter.

1. Open `res/drawable/ic_broken_image.xml`, and click the **Design** tab on the right. For the error image, you're using the broken-image icon that's available in the built-in icon library. This vector drawable uses the `android:tint` attribute to color the icon gray.



2. Open `res/drawable/loading_animation.xml`. This drawable is an animation that rotates an image drawable, `loading_img.xml`, around the center point. (You don't see the animation in the preview.)



3. Return to the `BindingAdapters.kt` file. In the `bindImage()` method, update the call to `imageView.load(imgUri)` to add a trailing lambda as follows: This code sets the placeholder loading image to use while loading (the `loading_animation` drawable). This code also sets an image to use if image loading fails (the `broken_image` drawable).

```
imageView.load(imgUri) {  
    placeholder(R.drawable.loading_animation)  
    error(R.drawable.ic_broken_image)  
}
```

4. The complete `bindImage()` method now looks like this:

```
@BindingAdapter("imageUrl")  
fun bindImage(imgView: ImageView, imgUrl: String?) {  
    imgUrl?.let {  
        val imgUri = imgUrl.toUri().buildUpon().scheme("https").build()  
        imageView.load(imgUri) {  
            placeholder(R.drawable.loading_animation)  
            error(R.drawable.ic_broken_image)  
        }  
    }  
}
```

5. Run the app. Depending on the speed of your network connection, you might briefly see the loading image as Coil downloads and displays the property image. But you won't see the broken-image icon yet, even if you turn off your network—you will fix that in the last task of the codelab.

2:57



1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16
17	18	19	20
21	22	23	24
25	26	27	28
29	30	31	32
33	34	35	36
37	38	39	40
41	42	43	44
45	46	47	48
49	50	51	52
53	54	55	56
57	58	59	60
61	62	63	64
65	66	67	68
69	70	71	72
73	74	75	76
77	78	79	80
81	82	83	84
85	86	87	88
89	90	91	92
93	94	95	96
97	98	99	100
101	102	103	104
105	106	107	108
109	110	111	112
113	114	115	116
117	118	119	120
121	122	123	124
125	126	127	128
129	130	131	132
133	134	135	136
137	138	139	140
141	142	143	144
145	146	147	148
149	150	151	152
153	154	155	156
157	158	159	160
161	162	163	164
165	166	167	168
169	170	171	172
173	174	175	176
177	178	179	180
181	182	183	184
185	186	187	188
189	190	191	192
193	194	195	196
197	198	199	200
201	202	203	204
205	206	207	208
209	210	211	212
213	214	215	216
217	218	219	220
221	222	223	224
225	226	227	228
229	230	231	232
233	234	235	236
237	238	239	240
241	242	243	244
245	246	247	248
249	250	251	252
253	254	255	256
257	258	259	260
261	262	263	264
265	266	267	268
269	270	271	272
273	274	275	276
277	278	279	280
281	282	283	284
285	286	287	288
289	290	291	292
293	294	295	296
297	298	299	300
301	302	303	304
305	306	307	308
309	310	311	312
313	314	315	316
317	318	319	320
321	322	323	324
325	326	327	328
329	330	331	332
333	334	335	336
337	338	339	340
341	342	343	344
345	346	347	348
349	350	351	352
353	354	355	356
357	358	359	360
361	362	363	364
365	366	367	368
369	370	371	372
373	374	375	376
377	378	379	380
381	382	383	384
385	386	387	388
389	390	391	392
393	394	395	396
397	398	399	400
401	402	403	404
405	406	407	408
409	410	411	412
413	414	415	416
417	418	419	420
421	422	423	424
425	426	427	428
429	430	431	432
433	434	435	436
437	438	439	440
441	442	443	444
445	446	447	448
449	450	451	452
453	454	455	456
457	458	459	460
461	462	463	464
465	466	467	468
469	470	471	472
473	474	475	476
477	478	479	480
481	482	483	484
485	486	487	488
489	490	491	492
493	494	495	496
497	498	499	500
501	502	503	504
505	506	507	508
509	510	511	512
513	514	515	516
517	518	519	520
521	522	523	524
525	526	527	528
529	530	531	532
533	534	535	536
537	538	539	540
541	542	543	544
545	546	547	548
549	550	551	552
553	554	555	556
557	558	559	560
561	562	563	564
565	566	567	568
569	570	571	572
573	574	575	576
577	578	579	580
581	582	583	584
585	586	587	588
589	590	591	592
593	594	595	596
597	598	599	600
601	602	603	604
605	606	607	608
609	610	611	612
613	614	615	616
617	618	619	620
621	622	623	624
625	626	627	628
629	630	631	632
633	634	635	636
637	638	639	640
641	642	643	644
645	646	647	648
649	650	651	652
653	654	655	656
657	658	659	660
661	662	663	664
665	666	667	668
669	670	671	672
673	674	675	676
677	678	679	680
681	682	683	684
685	686	687	688
689	690	691	692
693	694	695	696
697	698	699	700
701	702	703	704
705	706	707	708
709	710	711	712
713	714	715	716
717	718	719	720
721	722	723	724
725	726	727	728
729	730	731	732
733	734	735	736
737	738	739	740
741	742	743	744
745	746	747	748
749	750	751	752
753	754	755	756
757	758	759	760
761	762	763	764
765	766	767	768
769	770	771	772
773	774	775	776
777	778	779	780
781	782	783	784
785	786	787	788
789	790	791	792
793	794	795	796
797	798	799	800
801	802	803	804
805	806	807	808
809	810	811	812
813	814	815	816
817	818	819	820
821	822	823	824
825	826	827	828
829	830	831	832
833	834	835	836
837	838	839	840
841	842	843	844
845	846	847	848
849	850	851	852
853	854	855	856
857	858	859	860
861	862	863	864
865	866	867	868
869	870	871	872
873	874	875	876
877	878	879	880
881	882	883	884
885	886	887	888
889	890	891	892
893	894	895	896
897	898	899	900
901	902	903	904
905	906	907	908
909	910	911	912
913	914	915	916
917	918	919	920
921	922	923	924
925	926	927	928
929	930	931	932
933	934	935	936
937	938	939	940
941	942	943	944
945	946	947	948
949	950	951	952
953	954	955	956
957	958	959	960
961	962	963	964
965	966	967	968
969	970	971	972
973	974	975	976
977	978	979	980
981	982	983	984
985	986	987	988
989	990	991	992
993	994	995	996
997	998	999	1000



6. Revert the temporary changes you made in `overview/OverviewFragment.kt`. In the method `onCreateview()`, uncomment the line that inflates `FragmentOverviewBinding`. Delete or comment out the line that inflates `GridViewItemMBinding`.

```
val binding = FragmentOverviewBinding.inflate(inflater)
// val binding = GridViewItemMBinding.inflate(inflater)
```

4. Display a grid of images with a RecyclerView

Your app now loads a Mars photo from the internet. Using the data from the first `MarsPhoto` list item, you've created a `LiveData` property in the `ViewModel`, and you've used the image URL from that Mars photo data to populate an `ImageView`. But the goal is for your app to display a grid of images, so in this task you will use a `RecyclerView` with a `Grid` layout manager to display the grid of images.

Update the view model

In the previous task, in the `OverviewViewModel`, you have added an `LiveData` object called `_photos` that holds one `MarsPhoto` object—the first one in the response list from the web service. In this step, you will change this `LiveData` to hold the entire list of `MarsPhoto` objects.

1. Open `overview/OverviewViewModel.kt`.
2. Change the `_photos` type to be a list of `MarsPhoto` objects.

```
private val _photos = MutableLiveData<List<MarsPhoto>>()
```

3. Replace the backing property `photos` type to the `List<MarsPhoto>` type as well:

```
val photos: LiveData<List<MarsPhoto>> = _photos
```

4. Scroll down to the `try {}` block inside the `getMarsPhotos()` method. The `MarsApi.retrofitService.getPhotos()`

returns a list of `MarsPhoto` objects, you can just assign it to `_photos.value`.

```
_photos.value = MarsApi.retrofitService.getPhotos()
_status.value = "Success: Mars properties retrieved"
```

5. The entire `try/catch` block now looks like this:

```
try {
    _photos.value = MarsApi.retrofitService.getPhotos()
    _status.value = "Success: Mars properties retrieved"
} catch (e: Exception) {
    _status.value = "Failure: ${e.message}"
}
```

Grid Layout

The [GridLayoutManager](#) for RecyclerView lays out the data as a scrollable grid, as shown below.

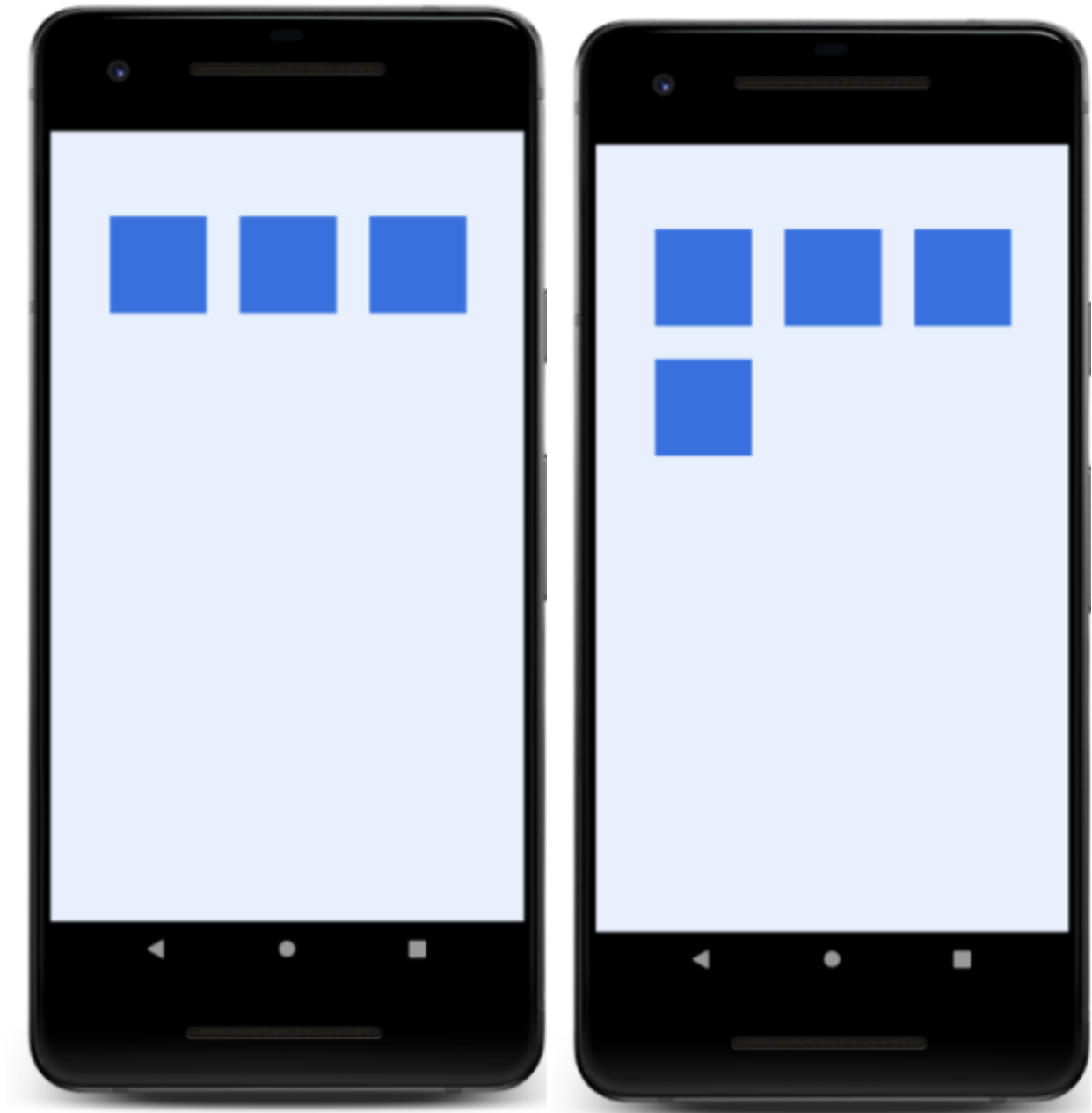


From a design perspective, Grid Layout is best for lists that can be represented as icons or images, such as lists within your Mars photo browsing app.

How Grid layout lays out items

Grid layout arranges items in a grid of rows and columns. Assuming vertical scrolling, by default, each item in a row takes up one "span." An item can occupy multiple spans. In the case below, one span is equivalent to the width of one column that is 3.

In the two examples shown below, each row is made up of three spans. By default, the `GridLayoutManager` lays out each item in one span until the span count, which you specify. When it reaches the span count, it wraps to the next line.



Add Recyclerview

In this step you will change the app's layout to use a recycler view with a grid layout, rather than the single image view.

1. Open `layout/grid_view_item.xml`. Remove the `viewModel` data variable.
2. Inside the `<data>` tag, add the following `photo` variable of the type `MarsPhoto`.

```
<data>
  <variable
    name="photo"
    type="com.example.android.marsphotos.network.MarsPhoto" />
</data>
```

3. In the `<ImageView>`, change the `app:imageUrl` attribute to refer to the image URL in the `MarsPhoto` object. These changes undo the temporary changes you made in this previous task.

```
app:imageUrl="@{photo.imgSrcUrl}"
```

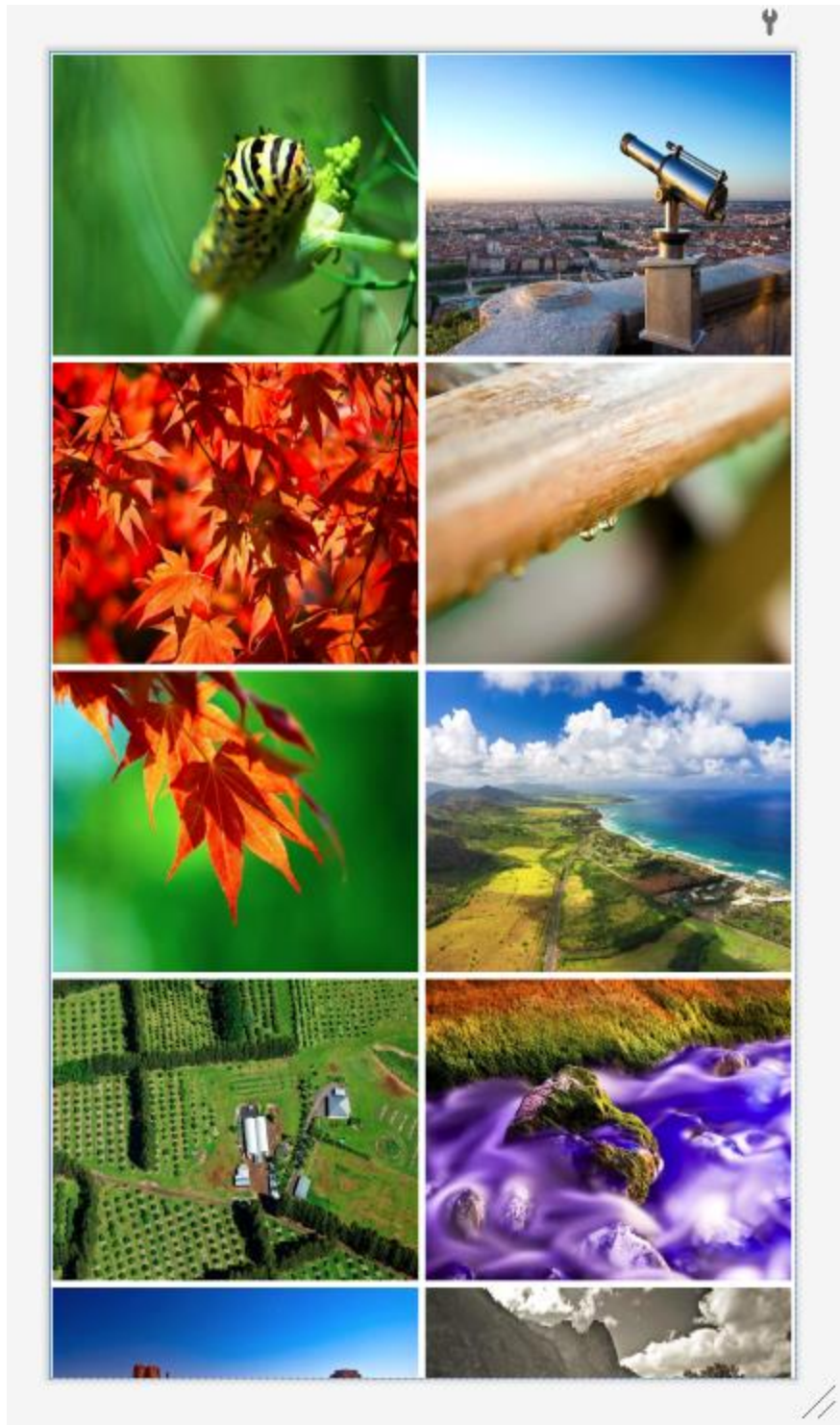
4. Open `layout/fragment_overview.xml`. Delete the entire `<TextView>` element.
5. Add the following `<RecyclerView>` element instead. Set the ID to `photos_grid`, width and height attributes to `0dp`, so it fills the parent `ConstraintLayout`. You will be using a Grid layout, so set the `layoutManager` attribute to `androidx.recyclerview.widget.GridLayoutManager`. Set a `spanCount` to 2 so you'll have two columns.

```
<androidx.recyclerview.widget.RecyclerView
  android:id="@+id/photos_grid"
  android:layout_width="0dp"
  android:layout_height="0dp"
  app:layoutManager=
    "androidx.recyclerview.widget.GridLayoutManager"
  app:layout_constraintBottom_toBottomOf="parent"
  app:layout_constraintLeft_toLeftOf="parent"
  app:layout_constraintRight_toRightOf="parent"
  app:layout_constraintTop_toTopOf="parent"
  app:spanCount="2" />
```

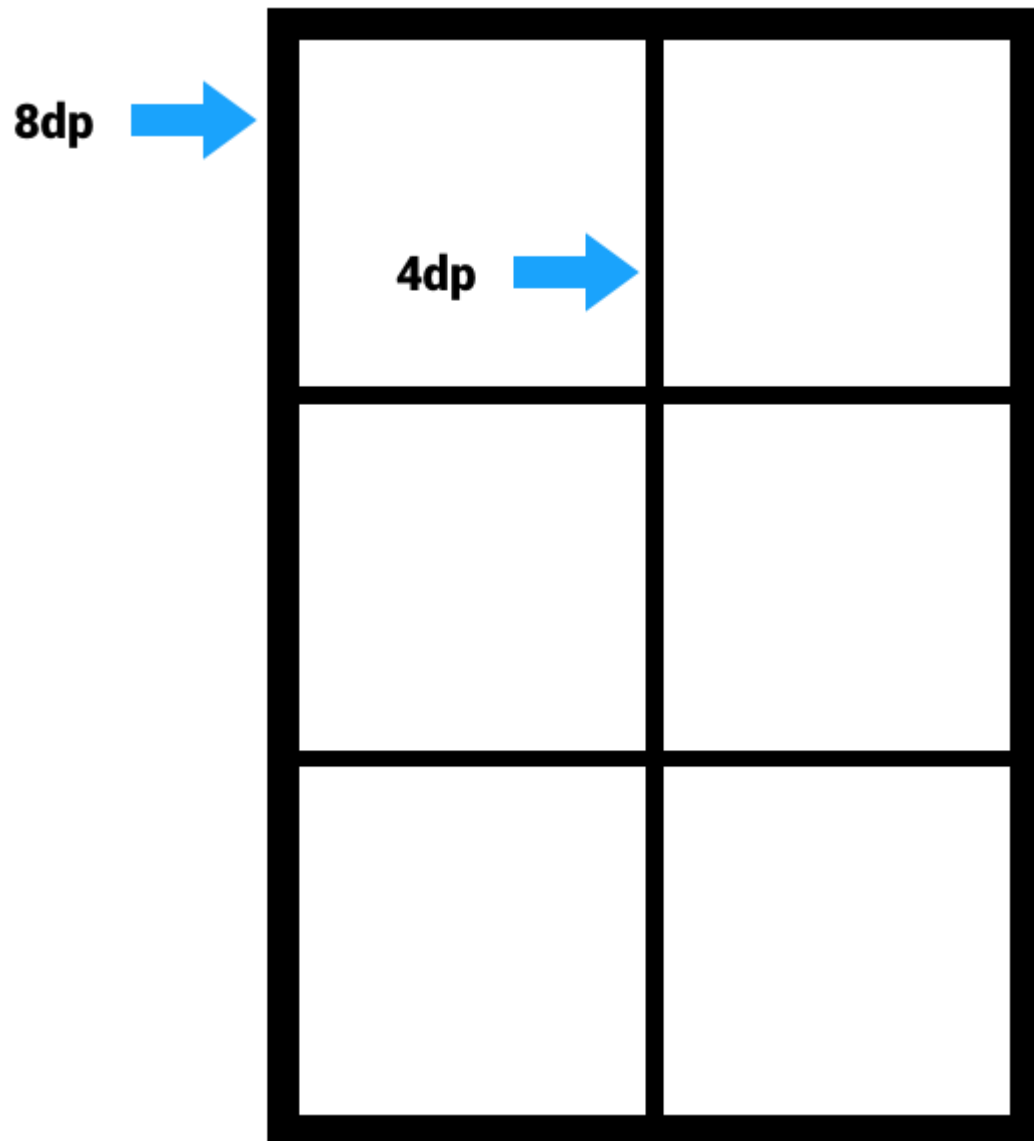
6. To see a preview of what the above code looks like in the **Design** view, use `tools:itemCount` to set the number of items displayed in our layout to 16. The `itemCount` attribute specifies the number of items the layout editor should render in the **Preview** window. Set the layout of list items to `grid_view_item` using `tools:listitem`.

```
<androidx.recyclerview.widget.RecyclerView
  ...
  tools:itemCount="16"
  tools:listitem="@layout/grid_view_item" />
```

7. Switch to **Design** view, you should see a preview something like the following screenshot. This does not look like Mars photos, but this will show you what your `recyclerview` grid layout will look like. Preview uses the padding and the `grid_view_item` layout for every single grid item the `recyclerview`.



8. According to the [Material design guidelines](#), you should have 8dp of space at the top, bottom, and sides of the list, and 4dp of space between the items. You can achieve this with a combination of padding in `fragment_overview.xml` layout and in `grid_view_item.xml` layout.



9. Open `layout/gridview_item.xml`. Notice the `padding` attribute, you already have 2dp of padding between the outside of the item and the content. This will get us 4dp of space between item content, and 2dp along the outside edges, which means we'll need an additional 6dp of padding on the outside edges to match the design guidelines.
10. Go back to `layout/fragment_overview.xml`. Add 6dp of padding for the `RecyclerView`, so you'll have 8dp on the outside and 4dp on the inside as the guidelines.

```
<androidx.recyclerview.widget.RecyclerView
    ...
    android:padding="6dp"
    ... />
```

11. The complete `<RecyclerView>` element should look like the following.

```

<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/photos_grid"
    android:layout_width="0dp"
    android:layout_height="0dp"
    android:padding="6dp"
    app:layoutManager=
        "androidx.recyclerview.widget.GridLayoutManager"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintLeft_toLeftOf="parent"
    app:layout_constraintRight_toRightOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:spanCount="2"
    tools:itemCount="16"
    tools:listitem="@layout/grid_view_item" />

```

Add the photo grid adapter

Now the `fragment_overview` layout has a `RecyclerView` with a grid layout. In this step, you'll bind the data retrieved from the web server to the `RecyclerView` through a `RecyclerView` adapter.

Note: This might be a good time to review the `RecyclerView` codelabs [in this pathway!](#)

ListAdapter (Refresher)

[ListAdapter](#) is a subclass of the [RecyclerView.Adapter](#) class for presenting List data in a `RecyclerView`, including computing diffs between Lists on a background thread.

In this app you will use the `DiffUtil` implementation in the `ListAdapter`. The advantage of using `DiffUtil` is every time some item in the `RecyclerView` is added, removed or changed, the whole list doesn't get refreshed. Only the items that have been changed are refreshed.

Add the [ListAdapter](#) to your app.

1. In the `overview` package, create a Kotlin class called `PhotoGridAdapter`.
2. Extend the `PhotoGridAdapter` class from `ListAdapter` with the constructor parameters shown below. The `PhotoGridAdapter` class extends `ListAdapter`, whose constructor needs the list item type, the view holder, and a `DiffUtil.ItemCallback` implementation.

```

class PhotoGridAdapter : ListAdapter<MarsPhoto,
    PhotoGridAdapter.MarsPhotoViewHolder>(DiffCallback) {
}

```

Import the `androidx.recyclerview.widget.ListAdapter` and `com.example.android.marsphoto.network.MarsPhoto` classes if requested. In the following steps, you'll implement the other missing implementations of this constructor that are producing errors.

3. To resolve the above errors, you will add the required methods in this step and implement them later in this task. Click on the `PhotoGridAdapter` class, click on the red bulb, from the dropdown menu select **Implement members**. In the popup displayed, select the `ListAdapter` methods, the `onCreateViewHolder()` and `onBindViewHolder()`. Android Studio will still show you errors which you will fix by the end of this task.

```
override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):  
PhotoGridAdapter.MarsPhotoViewHolder {  
    TODO("Not yet implemented")  
}  
  
override fun onBindViewHolder(holder: PhotoGridAdapter.MarsPhotoViewHolder,  
position: Int) {  
    TODO("Not yet implemented")  
}
```

To implement `onCreateViewHolder` and `onBindViewHolder` methods, you need `MarsPhotoViewHolder`, which you will add in the next step.

4. Inside the `PhotoGridAdapter`, add an inner class definition for `MarsPhotoViewHolder`, which extends `RecyclerView.ViewHolder`. You need the `GridViewItemBinding` variable for binding the `MarsPhoto` to the layout, so pass the variable into the `MarsPhotoViewHolder`. The base `ViewHolder` class requires a view in its constructor, you pass it the binding root view.

```
class MarsPhotoViewHolder(private var binding:  
    GridViewItemBinding):  
    RecyclerView.ViewHolder(binding.root) {  
}
```

Import `androidx.recyclerview.widget.RecyclerView` and `com.example.android.marsrealestate.databinding.GridViewItemBinding` if requested.

5. In `MarsPhotoViewHolder`, create a `bind()` method that takes a `MarsPhoto` object as an argument and sets `binding.property` to that object. Call `executePendingBindings()` after setting the property, which causes the update to execute immediately.

```
fun bind(MarsPhoto: MarsPhoto) {  
    binding.photo = MarsPhoto  
    binding.executePendingBindings()  
}
```

Note: This change may result in data-binding errors in Android Studio. To resolve those errors, you may need to clean and rebuild the app.

6. Still inside the `PhotoGridAdapter` class in `onCreateViewHolder()`, remove the `TODO` and add the line shown below. The `onCreateViewHolder()` method needs to return a new `MarsPhotoViewHolder`, created by inflating the `GridViewItemBinding` and using the `LayoutInflater` from your parent `ViewGroup` context.

```
return MarsPhotoViewHolder(GridviewItemBinding.inflate(
    LayoutInflater.from(parent.context)))
```

Import `android.view.LayoutInflater` if requested.

7. In the `onBindViewHolder()` method, remove the `TODO` and add the lines shown below. Here you call `getItem()` to get the `MarsPhoto` object associated with the current `RecyclerView` position, and then pass that property to the `bind()` method in the `MarsPhotoViewHolder`.

```
val marsPhoto = getItem(position)
holder.bind(marsPhoto)
```

8. Inside the `PhotoGridAdapter`, add a companion object definition for `DiffCallback`, as shown below.
The `DiffCallback` object extends `DiffUtil.ItemCallback` with the generic type of object you want to compare—`MarsPhoto`. You will compare two `Mars` photo objects inside this implementation.

```
companion object DiffCallback : DiffUtil.ItemCallback<MarsPhoto>() {
}
```

Import `androidx.recyclerview.widget.DiffUtil` when requested.

9. Press the red bulb to implement the comparator methods for the `DiffCallback` object, which are `areItemsTheSame()` and `areContentsTheSame()`.

```
override fun areItemsTheSame(oldItem: MarsPhoto, newItem: MarsPhoto): Boolean
{
    TODO("Not yet implemented")
}
```

```
override fun areContentsTheSame(oldItem: MarsPhoto, newItem: MarsPhoto):
Boolean {
    TODO("Not yet implemented") }
```

10. In the `areItemsTheSame()` method, remove the `TODO`. This method is called by `DiffUtil` to decide whether two objects represent the same `Item`. `DiffUtil` uses this method to figure out if the new `MarsPhoto` object is the same as the old `MarsPhoto` object. The ID of every `item(MarsPhoto object)` is unique. Compare the IDs of `oldItem` and `newItem`, and return the result.

```
override fun areItemsTheSame(oldItem: MarsPhoto, newItem: MarsPhoto): Boolean
{
    return oldItem.id == newItem.id
}
```

11. In `areContentsTheSame()`, remove the `TODO`. This method is called by `DiffUtil` when it wants to check whether two items have the same data. The important data in the

MarsPhoto is the image URL. Compare the URLs of `oldItem` and `newItem`, and return the result.

```
override fun areContentsTheSame(oldItem: MarsPhoto, newItem: MarsPhoto):  
Boolean {  
    return oldItem.imgSrcUrl == newItem.imgSrcUrl  
}
```

Make sure you are able to compile and run the app without any errors, but the emulator displays a blank screen. You have the `recyclerView` ready but there is no data passed to it, which you will implement in the next step.

Add the binding adapter and connect the parts

In this step, you will use a `BindingAdapter` to initialize the `PhotoGridAdapter` with the list of `MarsPhoto` objects. Using a `BindingAdapter` to set the `RecyclerView` data causes data binding to automatically observe the `LiveData` for the list of `MarsPhoto` objects. Then the binding adapter is called automatically when the `MarsPhoto` list changes.

1. Open `BindingAdapters.kt`.
2. At the end of the file, add a `bindRecyclerView()` method that takes a `RecyclerView` and a list of `MarsPhoto` objects as arguments. Annotate that method with a `@BindingAdapter` with `listData` attribute.

```
@BindingAdapter("listData")  
fun bindRecyclerView(recyclerView: RecyclerView,  
    data: List<MarsPhoto>?) {  
}
```

Import `androidx.recyclerview.widget.RecyclerView` and `com.example.android.marsphotos.network.MarsPhoto` if requested.

3. Inside the `bindRecyclerView()` function, cast `recyclerView.adapter` to `PhotoGridAdapter` and assign it to a new `val` property `adapter`.

```
val adapter = recyclerView.adapter as PhotoGridAdapter
```

4. At the end of `bindRecyclerView()` function, call `adapter.submitList()` with the Mars photos list data. This tells the `RecyclerView` when a new list is available.

```
adapter.submitList(data)
```

Import `com.example.android.marsrealestate.overview.PhotoGridAdapter` if requested.

5. The complete `bindRecyclerView` binding adapter should look like this:

```

@BindingAdapter("listData")
fun bindRecyclerView(recyclerView: RecyclerView,
                    data: List<MarsPhoto>?) {
    val adapter = recyclerView.adapter as PhotoGridAdapter
    adapter.submitList(data)
}

```

6. To connect everything together, open `res/layout/fragment_overview.xml`. Add the `app:listData` attribute to the `RecyclerView` element and set it to `viewModel.photos` using data binding. This is similar to what you have done for `ImageView` in a previous task.

```
app:listData="@{viewModel.photos}"
```

7. Open `overview/OverviewFragment.kt`. In `onCreateView()`, just before the `return` statement, initialize the `RecyclerView` adapter in `binding.photosGrid` to a new `PhotoGridAdapter` object.

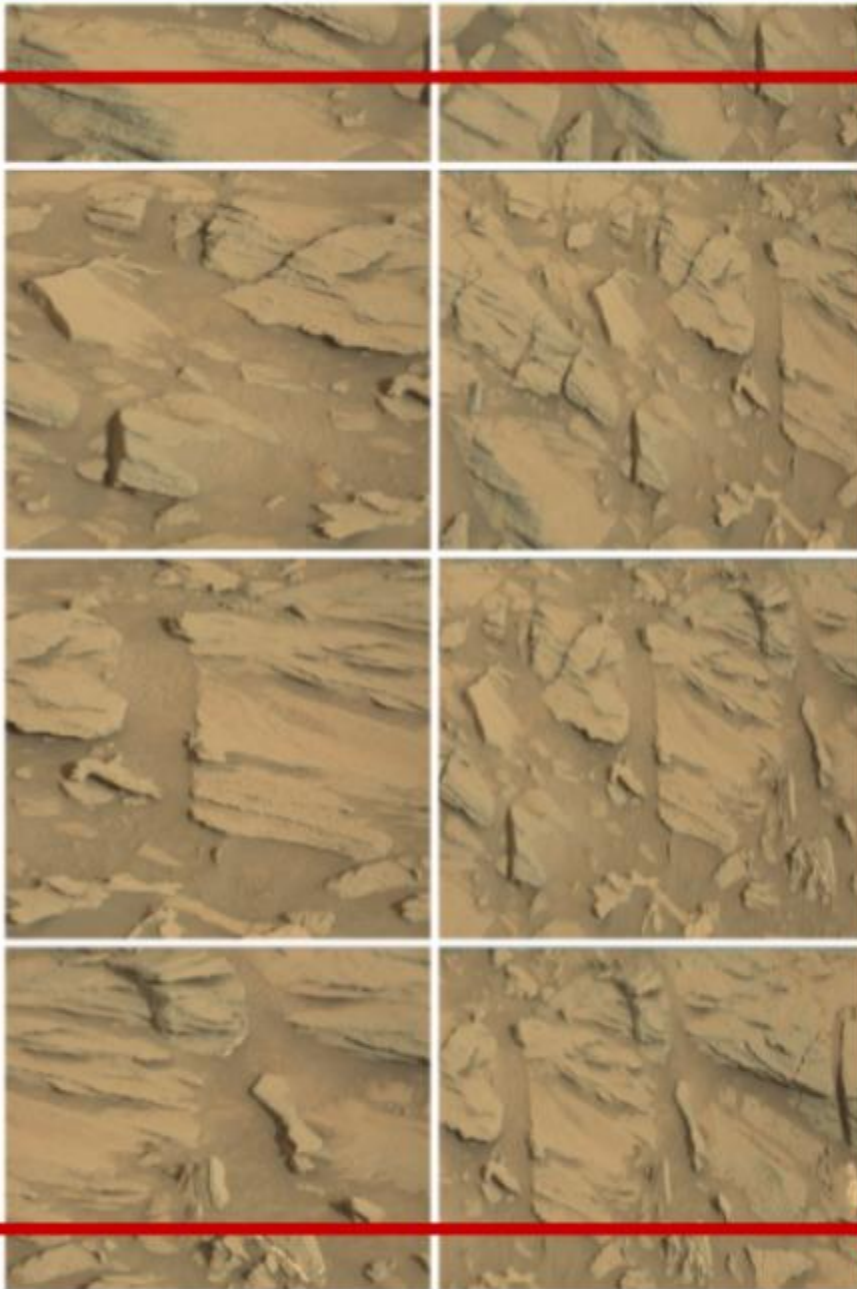
```
binding.photosGrid.adapter = PhotoGridAdapter()
```

8. Run the app. You should see a grid of scrolling Mars images. As you scroll to see new images but it looks a bit strange. The padding remains at the top and bottom of the `RecyclerView` as you're scrolling, so it never really looks like the list is being scrolled under the Action Bar.

12:23



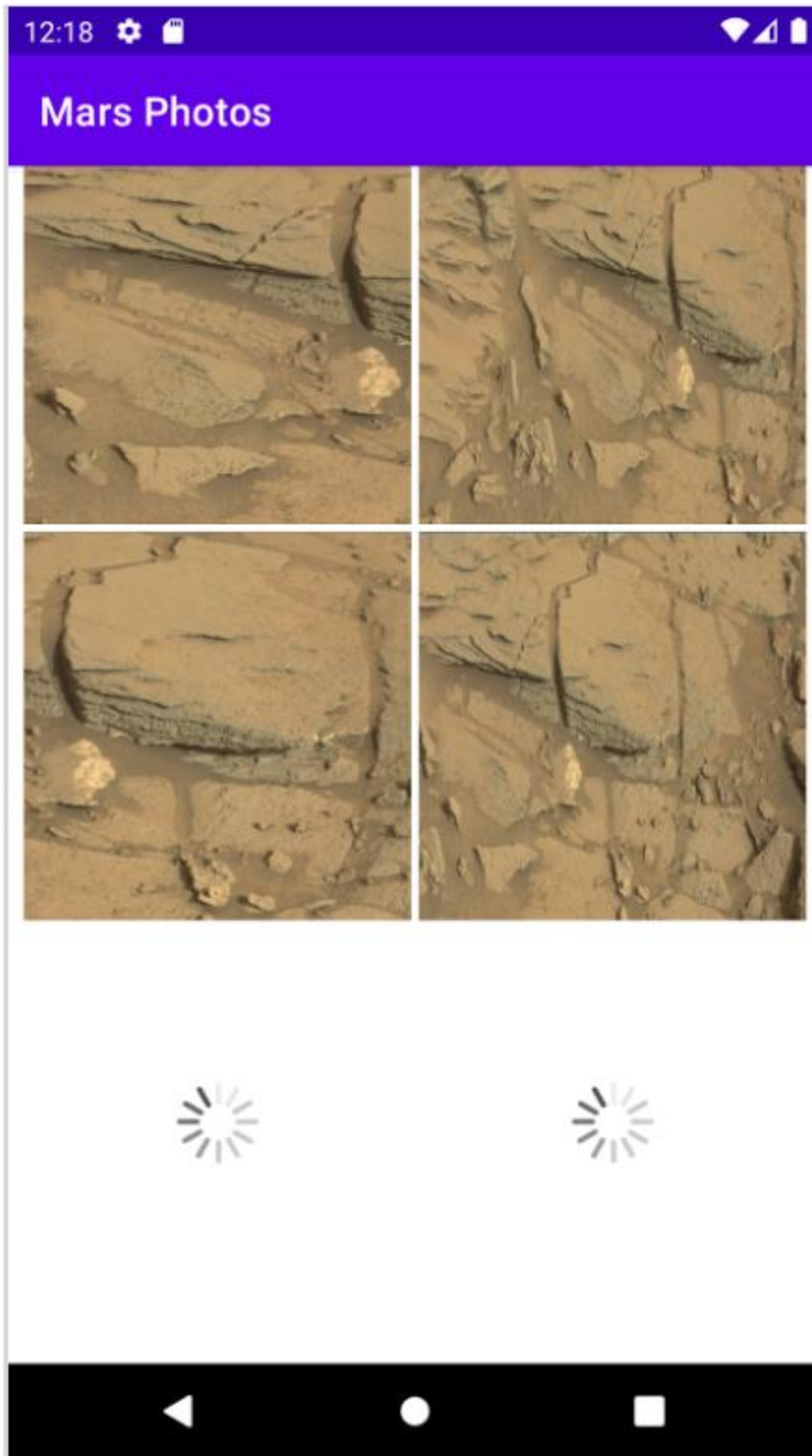
Mars Photos



9. To fix this, you need to tell the `RecyclerView` not to clip the inner contents to the padding using `android:clipToPadding` attribute. This makes it draw the scrolling view in the padding area. Go back to `layout/fragment_overview.xml`. Add `android:clipToPadding` attribute for the `RecyclerView`, set it to `false`.

```
<androidx.recyclerview.widget.RecyclerView
    ...
    android:clipToPadding="false"
    ... />
```

10. Run your app. Notice the app also shows the loading-progress icon before displaying the image itself, as expected. This is the placeholder loading image that you passed to the Coil image library.



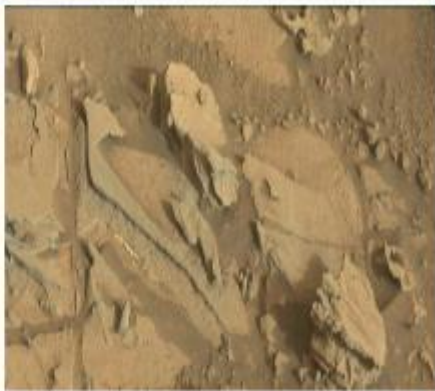
11. While the app is running, turn on airplane mode. Scroll the images in the emulator. Images that have not yet loaded appear as broken-image icons. This is the image

drawable that you passed to the Coil image library to display in case any network error or image could not be fetched.

12:16



Mars Photos



Congratulations, you are almost done. In the next final task you will further improve the user experience by adding more error handling to the app.

5. Add error handling in RecyclerView

The MarsPhotos app displays the broken-image icon when an image cannot be fetched. But when there's no network, the app shows a blank screen. You will verify the blank screen in the next step.

1. Turn on the airplane mode on your device or emulator. Run the app from the Android Studio. Notice the blank screen.

12:15



Mars Photos



This is not a great user experience. In this task, you will add a basic error handling, to give the user a better idea of what is happening. If the internet is not available, the app will show the connection-error icon and while the app is fetching the `MarsPhoto` list, the app will show the loading animation.

Add status to the ViewModel

In this task, you will create a property in the `OverviewViewModel` to represent the status of the web request. There are three states to consider—loading, success, and failure. The loading state happens while you're waiting for data. The success status is when we retrieve data successfully from the webservice. The failure status indicates any network or connection errors.

Enum Classes in Kotlin

To represent these three states in your application, you will use [enum](#). `enum` is short for enumeration, which means ordered listing of all the items in a collection. Each `enum` constant is an object of the `enum` class.

In Kotlin, an `enum` is a data type that can hold a set of constants. They are defined by adding the keyword `enum` in front of a class definition as shown below. Enum constants are separated with commas.

Definition:

```
enum class Direction {  
    NORTH, SOUTH, WEST, EAST  
}
```

Usage:

```
var direction : Direction = Direction.NORTH
```

As shown above, the `enum` objects can be referenced using the class name followed by a dot (`.`) operator and the name of the constant.

Note: Enum constants can be initialized with custom values using a constructor parameter in the class definition. This is beyond the scope of this codelab, please refer to the [Kotlin documentation](#) to learn more.

Add the `enum` class definition with the status values in the `ViewModel`.

1. Open `overview/OverviewViewModel.kt`. At the top of the file (after the imports, before the class definition), add an `enum` to represent all the available statuses:

```
enum class MarsApiStatus { LOADING, ERROR, DONE }
```

2. Scroll to the definition of `_status` and `status` properties, change the types from `String` to `MarsApiStatus`. `MarsApiStatus` is the enum class you defined in the previous step.

```
private val _status = MutableLiveData<MarsApiStatus>()

val status: LiveData<MarsApiStatus> = _status
```

3. In the `getMarsPhotos()` method, change the "Success: ..." string to the `MarsApiStatus.DONE` state, and the "Failure..." string to `MarsApiStatus.ERROR`.

```
try {
    _photos.value = MarsApi.retrofitService.getPhotos()
    _status.value = MarsApiStatus.DONE
} catch (e: Exception) {
    _status.value = MarsApiStatus.ERROR
}
```

4. Set the status to `MarsApiStatus.LOADING` above the `try {}` block. This is the initial status while the coroutine is running and you're waiting for data. The complete `viewModelScope.launch {}` block now looks like this:

```
viewModelScope.launch {
    _status.value = MarsApiStatus.LOADING
    try {
        _photos.value = MarsApi.retrofitService.getPhotos()
        _status.value = MarsApiStatus.DONE
    } catch (e: Exception) {
        _status.value = MarsApiStatus.ERROR
    }
}
```

5. After the error state in the `catch {}` block, set the `_photos` to an empty list. This clears the RecyclerView.

```
} catch (e: Exception) {
    _status.value = MarsApiStatus.ERROR
    _photos.value = listOf()
}
```

6. The complete `getMarsPhotos()` method should look like this:

```
private fun getMarsPhotos() {
    viewModelScope.launch {
        _status.value = MarsApiStatus.LOADING
        try {
            _photos.value = MarsApi.retrofitService.getPhotos()
            _status.value = MarsApiStatus.DONE
        } catch (e: Exception) {
            _status.value = MarsApiStatus.ERROR
            _photos.value = listOf()
        }
    }
}
```

```
}  
}
```

You have defined enum states for the status and set the loading state at the beginning of the coroutine, set done when your app is finished retrieving the data from the web server, and set error when there is an exception. In the next task you will use a binding adapter to display the corresponding icons.

Add a binding adapter for the status `ImageView`

You have set up the `MarsApiStatus` in the `OverviewViewModel`, using a set of enum states. In this step, you will make it appear in the app. You use a Binding adapter for an `ImageView`, to display icons for the loading and error states. When the app is in the loading state or the error state, the `ImageView` should be visible. When the app is done loading, the `ImageView` should be invisible.

1. Open `BindingAdapters.kt`, scroll to the end of the file to add another adaptor. Add a new binding adapter called `bindStatus()` that takes an `ImageView` and a `MarsApiStatus` value as arguments. Annotate the method with `@BindingAdapter` passing in the custom attribute `marsApiStatus` as parameter.

```
@BindingAdapter("marsApiStatus")  
fun bindStatus(statusImageView: ImageView,  
               status: MarsApiStatus?) {  
}
```

Import `com.example.android.marsrealestate.overview.MarsApiStatus` if requested.

2. Add a `when {}` block inside the `bindStatus()` method to switch between the different statuses.

```
when (status) {  
  
}
```

3. Inside the `when {}`, add a case for the loading state (`MarsApiStatus.LOADING`). For this state, set the `ImageView` to visible, and assign it the loading animation. This is the same animation drawable you used for Coil in the previous task.

```
when (status) {  
    MarsApiStatus.LOADING -> {  
        statusImageView.visibility = View.VISIBLE  
        statusImageView.setImageResource(R.drawable.loading_animation)  
    }  
}
```

Import `android.view.View` if requested.

4. Add a case for the error state, which is `MarsApiStatus.ERROR`. Similarly to what you did for the `LOADING` state, set the status `ImageView` to visible and use the `connection-error` drawable.

```
MarsApiStatus.ERROR -> {  
    statusImageView.visibility = View.VISIBLE  
    statusImageView.setImageResource(R.drawable.ic_connection_error)  
}
```

5. Add a case for the done state, which is `MarsApiStatus.DONE`. Here you have a successful response, so set the visibility of the status `ImageView` to `View.GONE` to hide it.

```
MarsApiStatus.DONE -> {  
    statusImageView.visibility = View.GONE  
}
```

You have set up the binding adapter for the status `Image` view, in the next step you will add an `Image` view which uses the new binding adapter.

Add the status `ImageView`

In this step, you will add the `Image` view in the `fragment_overview.xml` that will show the status you defined earlier.

1. Open `res/layout/fragment_overview.xml`. Inside the `ConstraintLayout`, below the `RecyclerView` element, add the `ImageView` shown below.

```
<ImageView  
    android:id="@+id/status_image"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    app:layout_constraintBottom_toBottomOf="parent"  
    app:layout_constraintLeft_toLeftOf="parent"  
    app:layout_constraintRight_toRightOf="parent"  
    app:layout_constraintTop_toTopOf="parent"  
    app:marsApiStatus="@{viewModel.status}" />
```

The above `ImageView` has the same constraints as the `RecyclerView`. However, the width and height use `wrap_content` to center the image rather than stretch the image to fill the view. Also notice the `app:marsApiStatus` attribute set to `viewModel.status`, which calls your `BindingAdapter` when the `status` property in the `ViewModel` changes.

2. To test the above code, simulate the network connection error by turning on airplane mode in your emulator or device. Compile and run the app, and notice that the error image appears:

10:46



Mars Photos



3. Tap the Back button to close the app, and turn off airplane mode. Use the recents screen to return the app. Depending on the speed of your network connection, you may see an extremely brief loading spinner when the app queries the web service before the images begin to load.

Congratulations on completing this codelab and building out the MarsPhotos app! It's time to show off your app with real life Mars pictures with your family and friends.

6. Solution code

The solution code for this codelab is in the project shown below. Use the **main** branch to pull or download the code.

Solution Code URL:

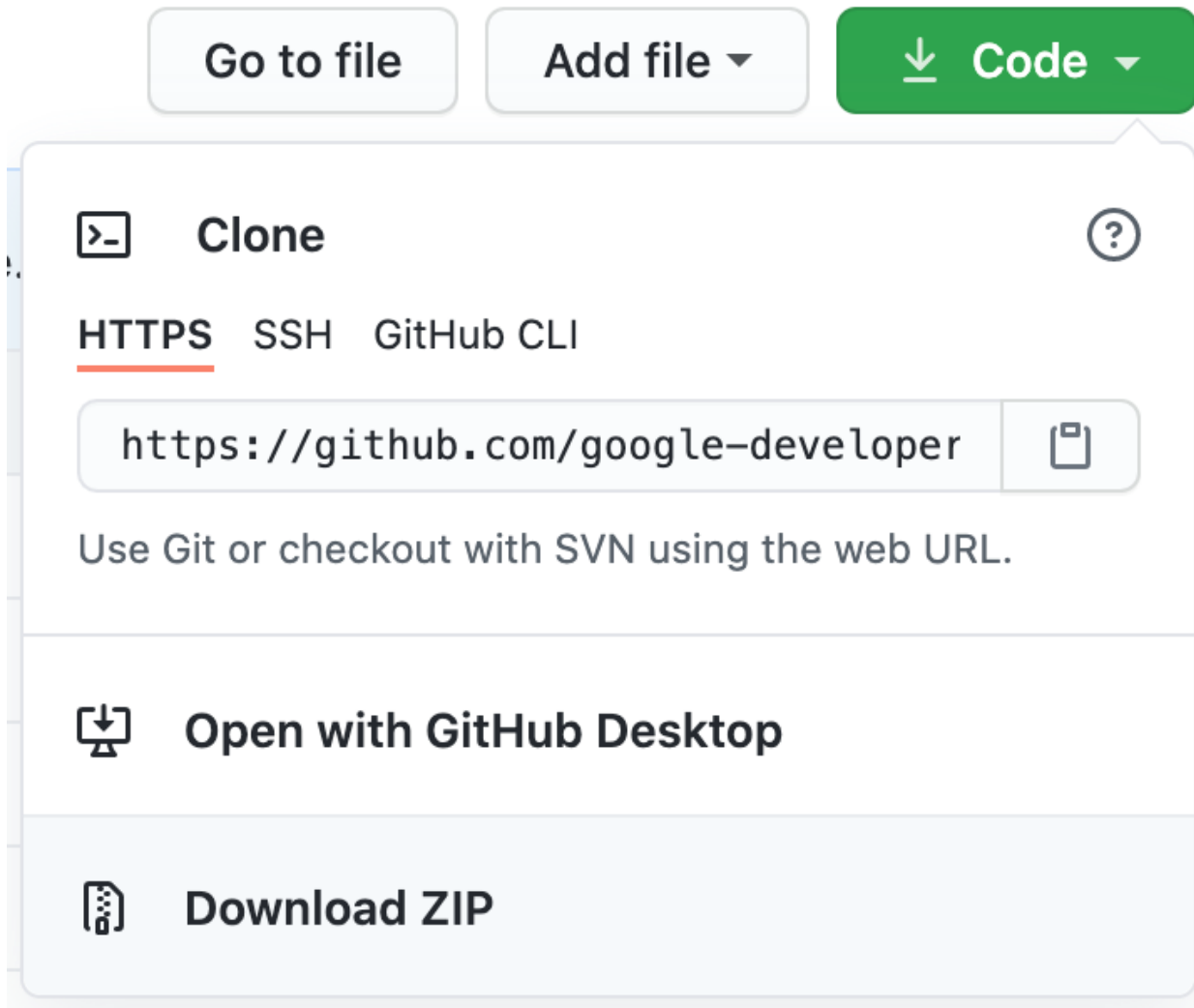
<https://github.com/google-developer-training/android-basics-kotlin-mars-photos-app/tree/main>

Branch name: main

To get the code for this codelab and open it in Android Studio, do the following.

Get the code

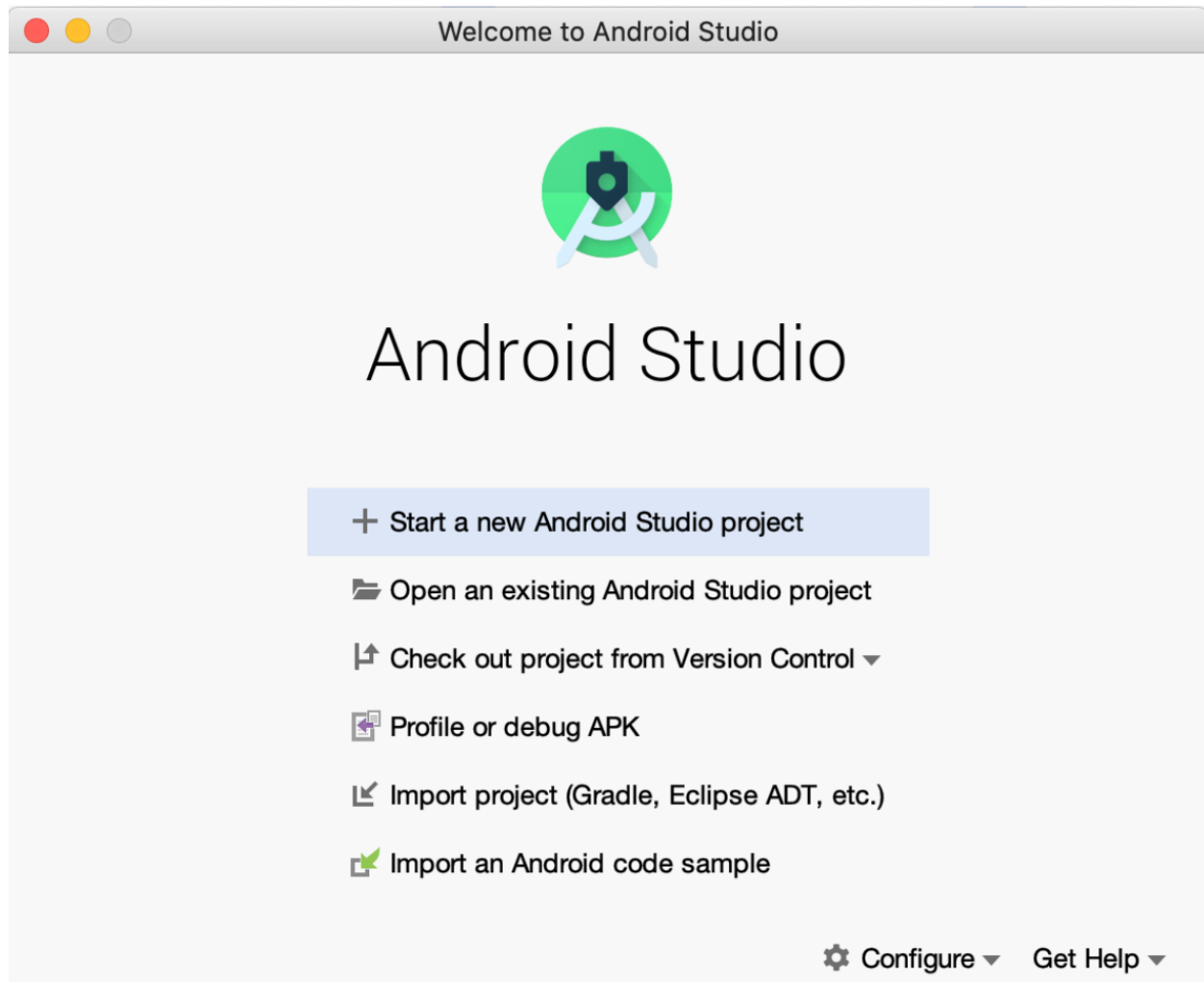
1. Click on the provided URL. This opens the GitHub page for the project in a browser.
2. On the GitHub page for the project, click the **Code** button, which brings up a dialog.



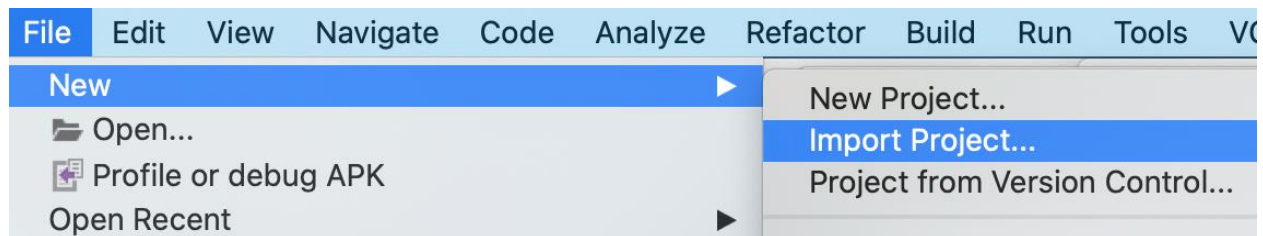
3. In the dialog, click the **Download ZIP** button to save the project to your computer. Wait for the download to complete.
4. Locate the file on your computer (likely in the **Downloads** folder).
5. Double-click the ZIP file to unpack it. This creates a new folder that contains the project files.

Open the project in Android Studio

1. Start Android Studio.
2. In the **Welcome to Android Studio** window, click **Open an existing Android Studio project**.



Note: If Android Studio is already open, instead, select the **File > New > Import Project** menu option.



3. In the **Import Project** dialog, navigate to where the unzipped project folder is located (likely in your **Downloads** folder).
4. Double-click on that project folder.
5. Wait for Android Studio to open the project.



6. Click the **Run** button to build and run the app. Make sure it builds as expected.

7. Browse the project files in the **Project** tool window to see how the app is set-up.

7. Summary

- The [Coil](#) library simplifies the process of managing images, such as download, buffer, decode, and cache images in your app.
- [Binding adapters](#) are extension methods that sit between a view and that view's bound data. Binding adapters provide custom behavior when the data changes, for example, to call Coil to load an image from a URL into an `ImageView`.
- Binding adapters are extension methods annotated with the `@BindingAdapter` annotation.
- To display a grid of images, use a `RecyclerView` with a `GridLayoutManager`.
- To update the list of properties when it changes, use a binding adapter between the `RecyclerView` and the layout.

8. Learn more

Android developer documentation:

- [ViewModel Overview](#)
- [LiveData Overview](#)
- [Coroutines, official documentation](#)
- [Binding adapters](#)

Other:

- [Coil](#)
- [Binding adapters with Kotlin](#)